Zephyr Project

IT-Schulungen.com
Technology & Transformation

Vom Quelltext zum ausführbaren Programm

Build Pipeline und native_sim Artefakte

1.06

Der Build-Weg in fünf Stationen

Von den Eingaben bis zum ausführbaren Linux-Programm

01

Eingaben

Anwendung, Ziel, Kconfig und Devicetree

02

Konfiguration

CMake erzeugt das konkrete Build-System

03

Übersetzen

Ninja steuert Compiler und Generatoren

04

Linken

Kernel, Anwendung und Modelle verbinden

05

Prüfen

Artefakte lesen und Altlasten vermeiden

Ziel: den Build-Weg erklären und native_sim klar von einer MCU-Emulation unterscheiden.

Vier Inputs bestimmen den Build

BUILD-KONTEXT

```
Anwendung + Board-Ziel + Konfiguration + Devicetree
```

Anwendung	Quellcode, CMakeLists.txt und anwendungsspezifische Dateien
Board-Ziel	native_sim oder native_sim/native/64 wählt Plattform und Modelle
Konfiguration	prj.conf und Kconfig wählen Funktionen und Abhängigkeiten
Devicetree	Boardbeschreibung und Overlays liefern Hardwareinformationen

Ändert sich eine Eingabe, muss das Build-System die wirksame Kombination neu bewerten.

Das Ziel für native_sim wählen

TERMINAL

```
$ west build -b native_sim app


# 64-Bit-Variante bei Hostbedarf
$ west build -b native_sim/native/64 app
```

01 **native_sim**

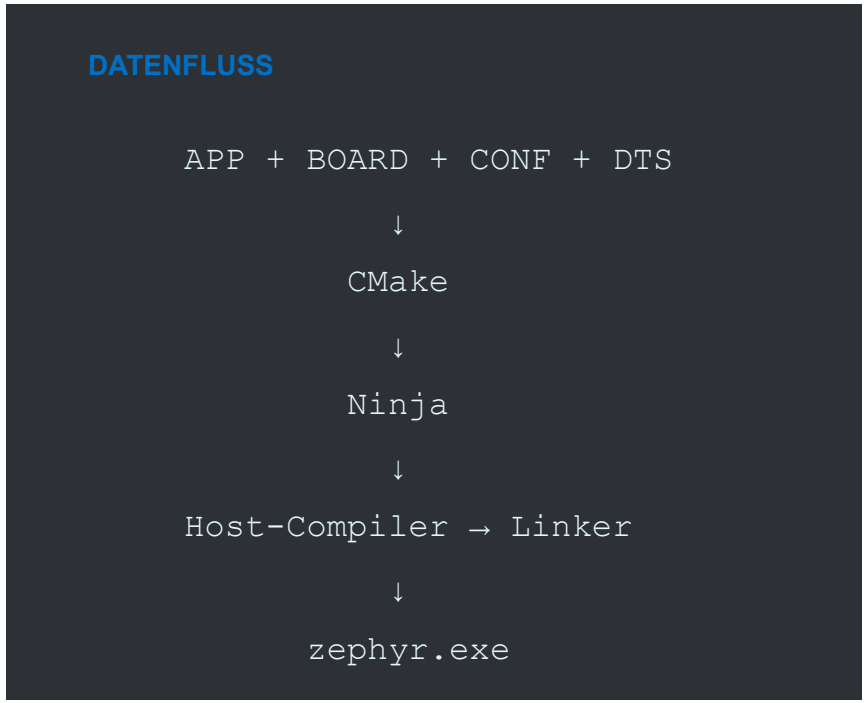
Bevorzugtes Trainingsziel für den normalen Host-Build.

02 **native/64**

Alternative bei fehlenden 32-Bit-Hostbibliotheken oder passender Umgebung.

Das Board-Ziel bestimmt POSIX-Architektur, Host-Eigenschaften und eingebundene Simulationsmodelle.

Die Build-Pipeline im Überblick



01 Konfigurieren

CMake, Kconfig und Devicetree erzeugen den Build-Plan.

02 Übersetzen

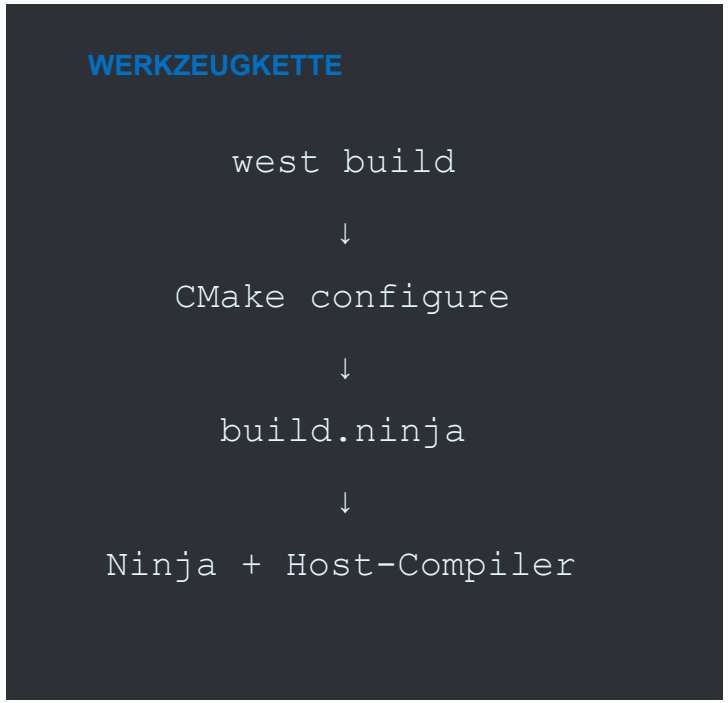
Compiler baut Anwendung, Kernel, Subsysteme und Treiber.

03 Linken

Linker verbindet Objekte und Hostmodelle zu zephyr.exe.

west build orchestriert die Stationen, ersetzt aber nicht ihre jeweils eigene Aufgabe.

CMake plant, Ninja führt aus



01 CMake

Wertet Anwendung und Zephyr-Buildlogik aus und erzeugt den konkreten Build-Plan.

02 Ninja

Führt Abhängigkeiten inkrementell aus und startet Generatoren, Compiler und Linker.

CMake entscheidet, was gebaut wird; Ninja entscheidet effizient, was erneut ausgeführt werden muss.

Kconfig, Devicetree und generierter Code

01	`Kconfig`	wählt Funktionen, Treiber und Quelltexte über Abhängigkeiten aus
02	`Devicetree`	beschreibt Geräte und erzeugt Hardwareinformationen sowie Makros
03	`Generierte Dateien`	verbinden Konfiguration und Hardwarebeschreibung mit dem C-Code

Der Compiler sieht nicht die Eingabedateien direkt, sondern ihre aufgelösten und generierten Ergebnisse.

Übersetzen und Linken

TERMINAL

```
$ west build -b native_sim app
[... CMake ...]
[... Generating ...]
[... Building ...]
[... Linking zephyr.exe ...]
```

01 Host-Compiler

Übersetzt Anwendung, Zephyr-Kernel, Subsysteme, Treiber und Simulator-Modelle.

02 Linker

Erzeugt aus allen Objekten das ausführbare Linux-Programm zephyr.exe.

Das Build-Log zeigt Konfiguration, Generierung, Übersetzung und Linken als getrennte Phasen.

Build-Artefakte lesen

BUILD/ZEPHYR

Die erzeugten Dateien erklären den tatsächlich wirksamen Build.

`.config`	aufgelöste Kconfig-Einstellungen
`zephyr.dts`	zusammengeführter Devicetree
`generated/`	generierte Header, Konstanten und Schnittstellen
`zephyr.map`	Linker-Sicht auf Symbole und Speicherbereiche
`zephyr.exe`	direkt auf dem Linux-Host ausführbares Ergebnis

Artefakte sind Belege: Sie zeigen, welche Konfiguration, Hardwarebeschreibung und Symbole verwendet wurden.

Firmware-Image oder natives Hostprogramm?

Ausführung	Firmware läuft auf Zielhardware; native_sim läuft als Linux-Prozess
Prozessor	Firmware nutzt die Ziel-ISA; native_sim wird für die Host-CPU gebaut
Peripherie	Firmware greift auf reale Geräte zu; native_sim nutzt Modelle und Host-Schnittstellen
Ergebnis	Firmware liefert Image-Formate; native_sim liefert zephyr.exe
Wichtige Grenze	zephyr.exe emuliert keine konkrete MCU und beweist kein reales Hardware-Timing

native_sim führt viel Zephyr-Code nativ aus, bildet aber keine vollständige Zielhardware nach.

Wann ist ein Pristine Build nötig?

AUSLÖSER

Build-Kontext ändert sich

Anderes Board-Ziel

Starke Konfigurationsänderung

Geänderte Toolchain

Verdächtiger Cache-Zustand

RISIKO

Veraltete Artefakte

CMake-Cache passt nicht

Generierte Dateien sind alt

Fehler wirken widersprüchlich

LÖSUNG

Sauber neu konfigurieren

west build -p always ...

oder gezielt -p pristine

Board und Anwendung erneut prüfen

Pristine behebt Zustandsprobleme, ersetzt aber keine Analyse echter Quell- oder Konfigurationsfehler.

Build-Weg verstanden

EINGABEN

Vier Entscheidungen

Anwendung

Board-Ziel

Kconfig

Devicetree

TRANSFORMATION

Vier Phasen

Konfigurieren

Generieren

Übersetzen

Linken

ERGEBNIS

Drei Nachweise

zephyr.exe erklären

.config und zephyr.dts finden

Grenze zur MCU benennen

Lernziel: Den Weg vom Quelltext zu zephyr.exe in eigenen Worten und ohne Werkzeugnamen-Liste erklären.